

Clase 8.1

Cierre del curso

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Menti*#Hablemos sobre la EP2*

Agenda

✨ **Grafos heterogéneos**

Features de arcos

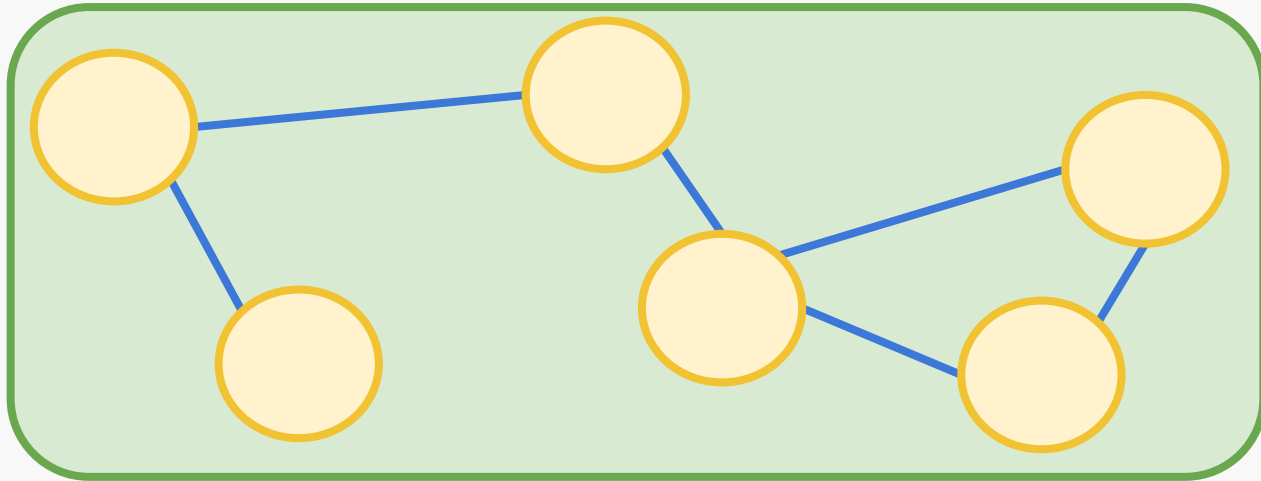
Algunas recomendaciones

Revisitando aplicaciones

Cierre

Componentes de una red

- **Objetos**: nodos, vértices (N) N (conjunto de nodos)
- **Interacciones**: arcos, aristas E (conjunto de aristas)
- **Sistema**: red, grafo $G(N, E)$ (grafo)



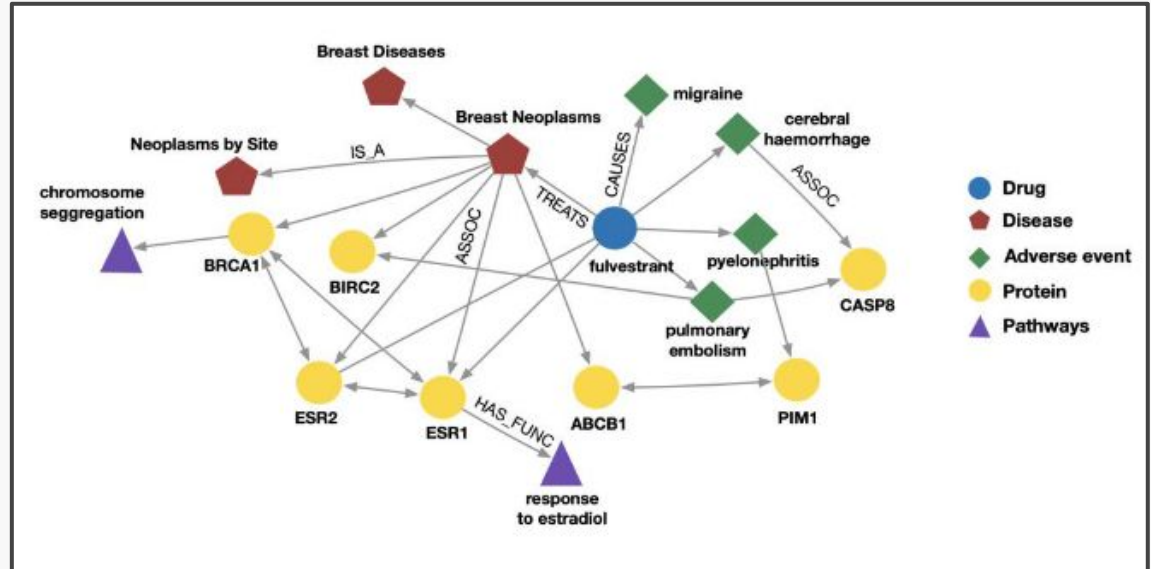
Grafos heterogéneos

Un grafo heterogéneo se puede definir como $G = (V, E, R, T)$, donde:

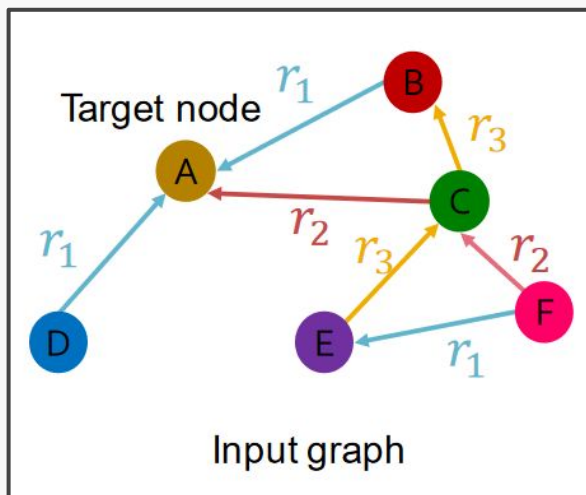
- V es el conjunto de nodos
- E son las relaciones entre nodos: (v_1, r, v_2)
- T indica el tipo de nodo: $T(v)$
- R es el conjunto de tipos de relaciones (r es parte de R)

Ejemplo de grafo heterogéneo

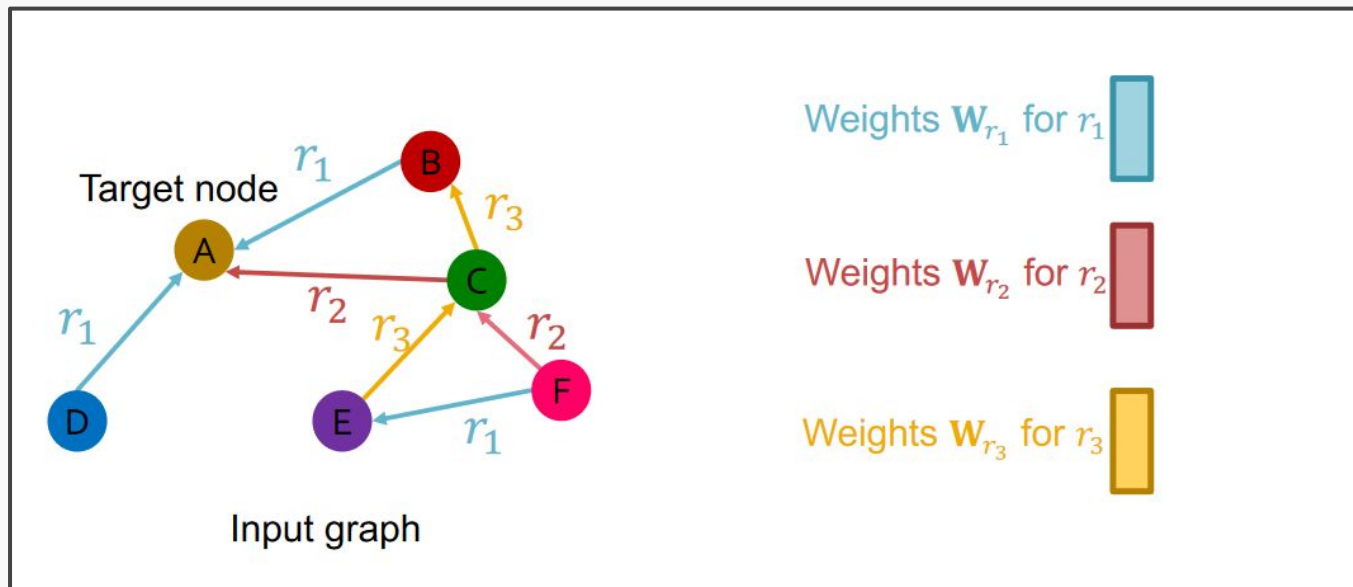
- **Nodo:** Migraña
- **Arco:** (fulvestrant, Treats, Breast Neoplasms)
- **Tipo de nodo:** Proteína
- **Tipo de arco:** "Causa"



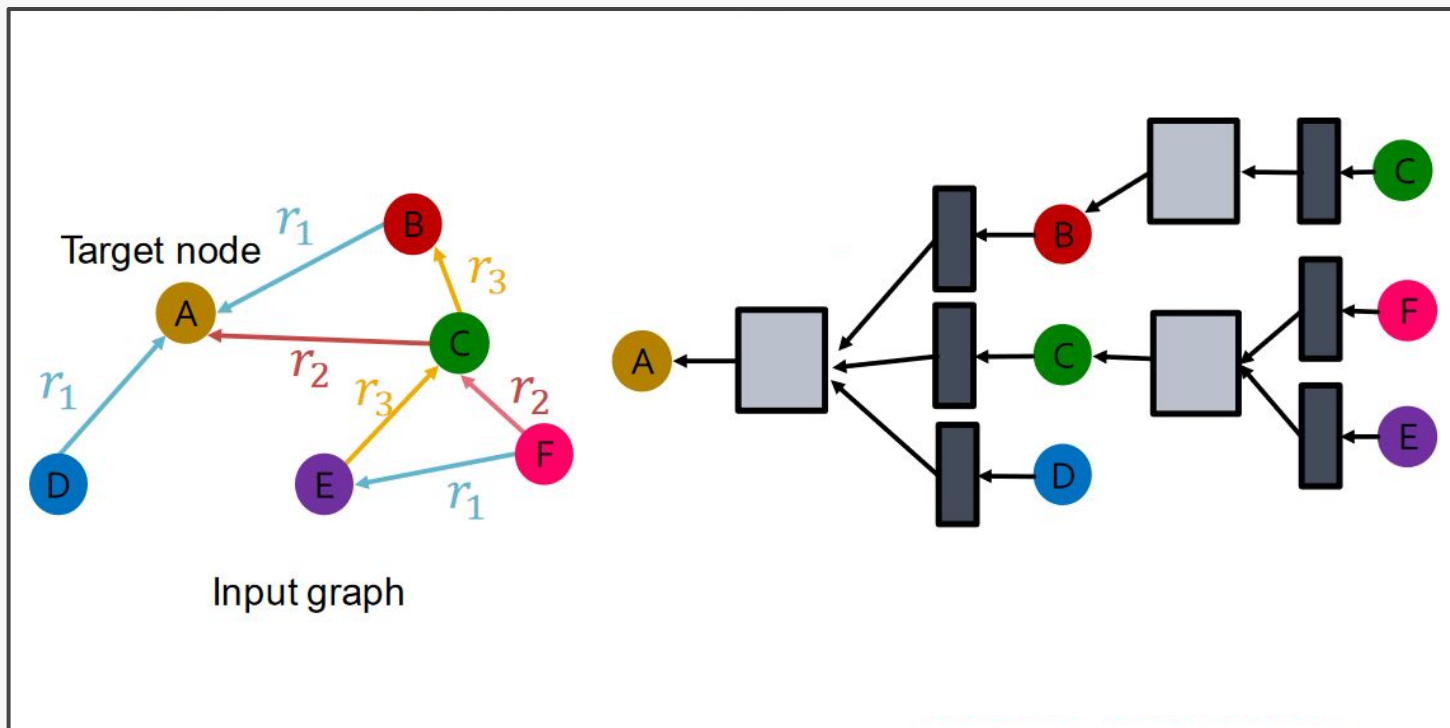
Distintos tipos de relaciones



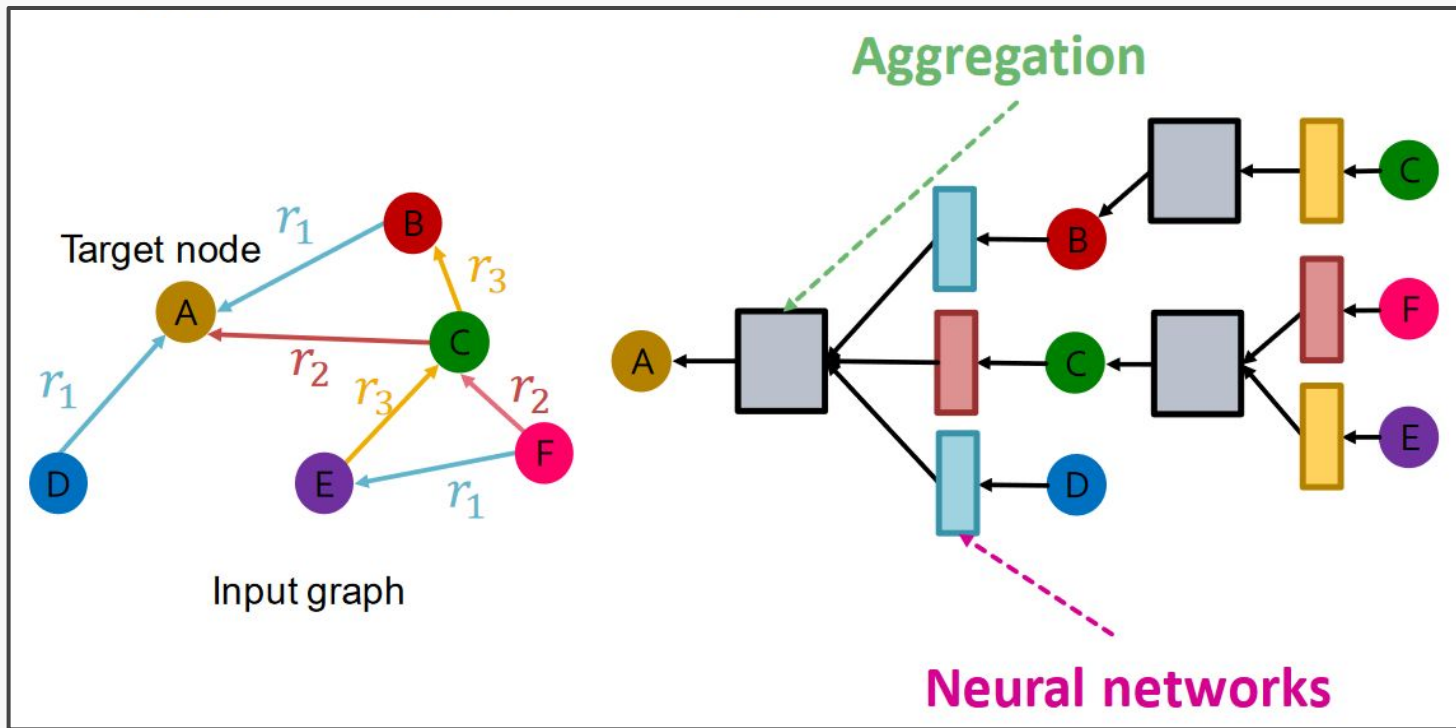
Distintos tipos de relaciones



Distintos tipos de relaciones



Distintos tipos de relaciones

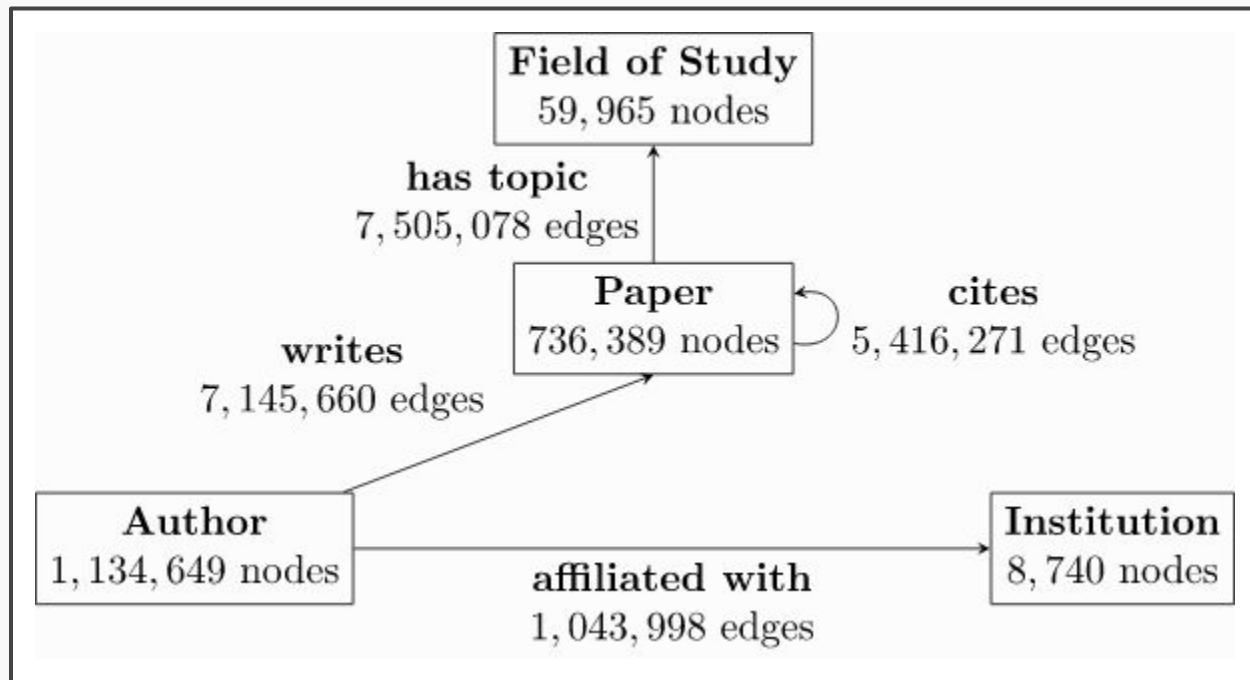


Distintos tipos de relaciones

Es exactamente lo mismo que la GCN que ya vimos, pero ocupamos un set de pesos para cada tipo de relación:

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\sum_{r \in R} \sum_{u \in N_v^r} \frac{1}{c_{v,r}} \mathbf{w}_r^{(l)} \mathbf{h}_u^{(l)} + \mathbf{w}_0^{(l)} \mathbf{h}_v^{(l)} \right)$$

Grafos heterogéneos



Agenda

Grafos heterogéneos



Features de arcos

Algunas recomendaciones

Revisitando aplicaciones

Cierre

Utilizamos una capa para las *features* de arcos

Así como utilizamos capas en el paso de mensajes de los nodos, podemos procesar las features de los arcos

```
1 class EdgeNetwork(layers.Layer):
2     # ...
3
4     def call(self, inputs):
5         atom_features, bond_features, pair_indices = inputs
6
7         # Apply linear transformation to bond features
8         bond_features = tf.matmul(bond_features, self.kernel) + self.bias
9
10        # Reshape for neighborhood aggregation later
11        bond_features = tf.reshape(bond_features, (-1, self.atom_dim, self.atom_dim))
12
13        # Obtain atom features of neighbors
14        atom_features_neighbors = tf.gather(atom_features, pair_indices[:, 1])
15        atom_features_neighbors = tf.expand_dims(atom_features_neighbors, axis=-1)
16
17        # Apply neighborhood aggregation
18        transformed_features = tf.matmul(bond_features, atom_features_neighbors)
19        transformed_features = tf.squeeze(transformed_features, axis=-1)
20        aggregated_features = tf.math.unsorted_segment_sum(
21            transformed_features,
22            pair_indices[:, 0],
23            num_segments=tf.shape(atom_features)[0],
24        )
25        return aggregated_features
```

Utilizamos una capa para las *features* de arcos

Luego, podemos utilizar ese *embedding* procesado en otras partes de nuestro modelo

```
1 class MessagePassing(layers.Layer):
2     # ...
3
4     def call(self, inputs):
5         atom_features, bond_features, pair_indices = inputs
6
7         # Pad atom features if number of desired units exceeds atom_features dim.
8         # Alternatively, a dense layer could be used here.
9         atom_features_updated = tf.pad(atom_features, [(0, 0), (0, self.pad_length)])
10
11        # Perform a number of steps of message passing
12        for i in range(self.steps):
13            # Aggregate information from neighbors
14            atom_features_aggregated = self.message_step(
15                [atom_features_updated, bond_features, pair_indices]
16            )
17
18            # Update node state via a step of GRU
19            atom_features_updated, _ = self.update_step(
20                atom_features_aggregated, atom_features_updated
21            )
22        return atom_features_updated
```

Agenda

Grafos heterogéneos

Features de arcos



Algunas recomendaciones

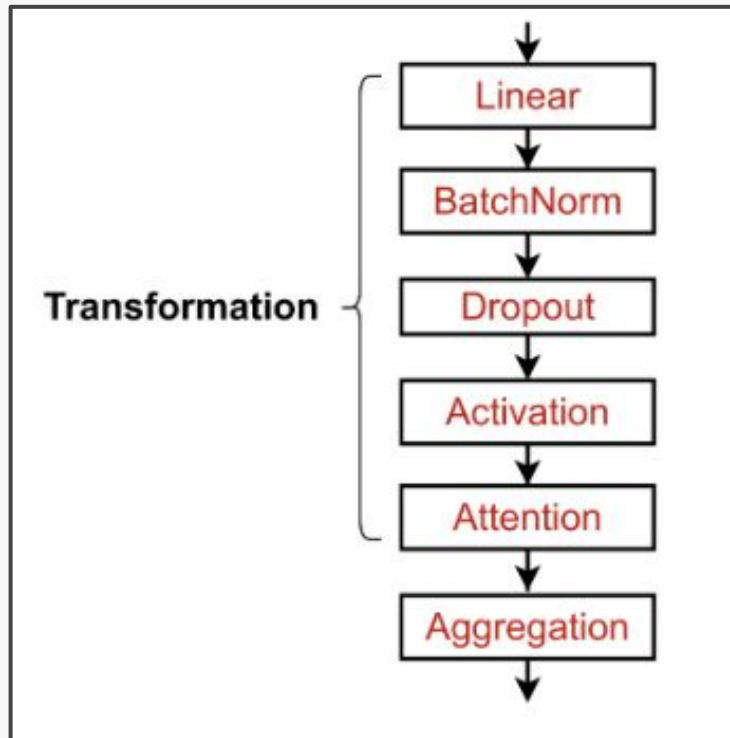
Revisitando aplicaciones

Cierre

En la práctica

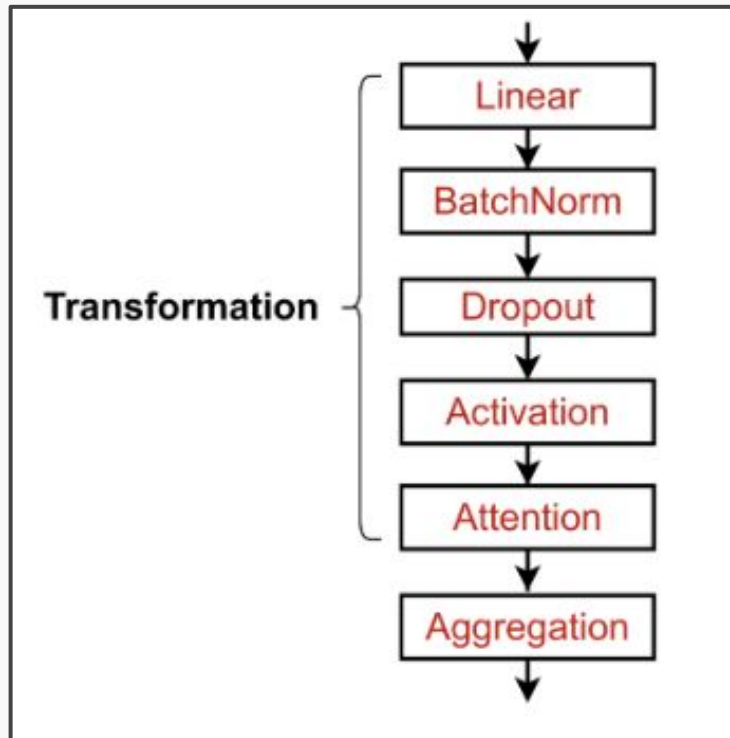
En la práctica, estas aproximaciones “clásicas” son un buen punto de inicio:

- Podemos mejorar la performance utilizando otros elementos de NNs en conjunto
- En concreto, podemos usar módulos modernos de DL en nuestras GNNs para mejorar nuestros modelos



En la práctica

- **Batch normalization:** Recentrar y escalar los embeddings (media cero, varianza uno)
- **Dropout:** Regularizar un modelo apagando neuronas al azar
- **Attention/gating**
- Y cualquier otro módulo



Algunas recomendaciones

1. Diseñar la tarea, antes que el modelo
2. Empiecen pequeño y simple: construyan un *baseline*
3. Agreguen solo la complejidad necesaria (tanto en datos como modelos)
4. Familiarizarse con la librería: sigan tutoriales, ejemplos, documentación, etc.
5. Prestar atención al proceso de entrenamiento (particiones, aumentación, etc.)

Agenda

Grafos heterogéneos

Features de arcos

Algunas recomendaciones



Revisitando aplicaciones

Cierre

Clase 1.3

Aplicaciones reales

Teoría de Grafos para Machine Learning

Revisitemos esta clase

Agenda

Grafos heterogéneos

Features de arcos

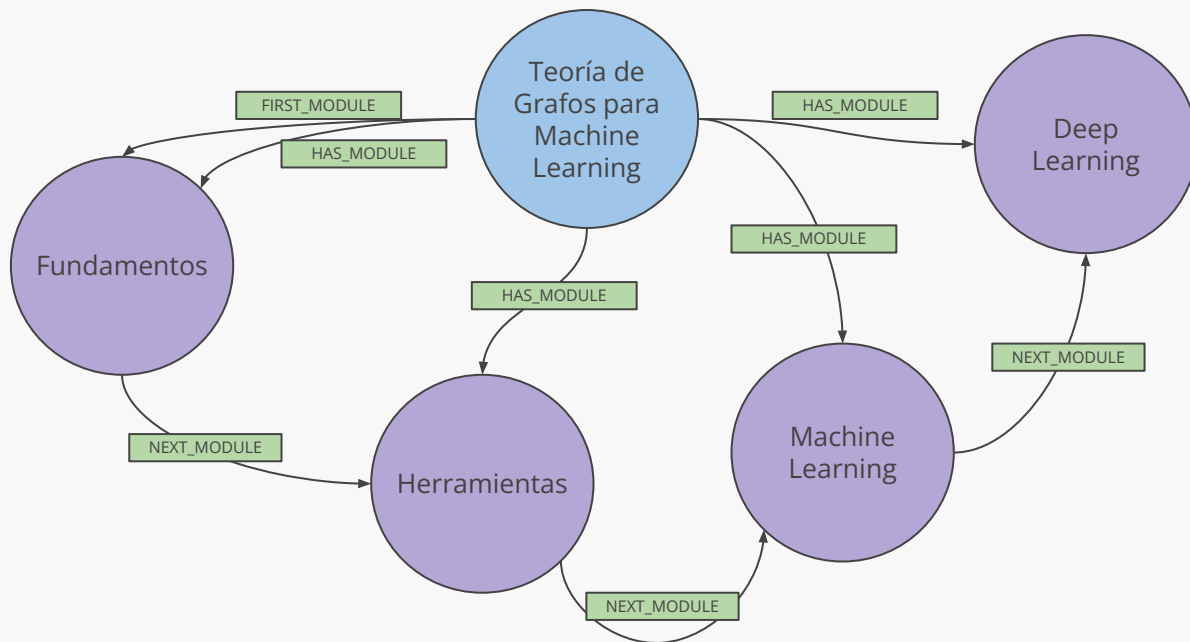
Algunas recomendaciones

Revisitando aplicaciones

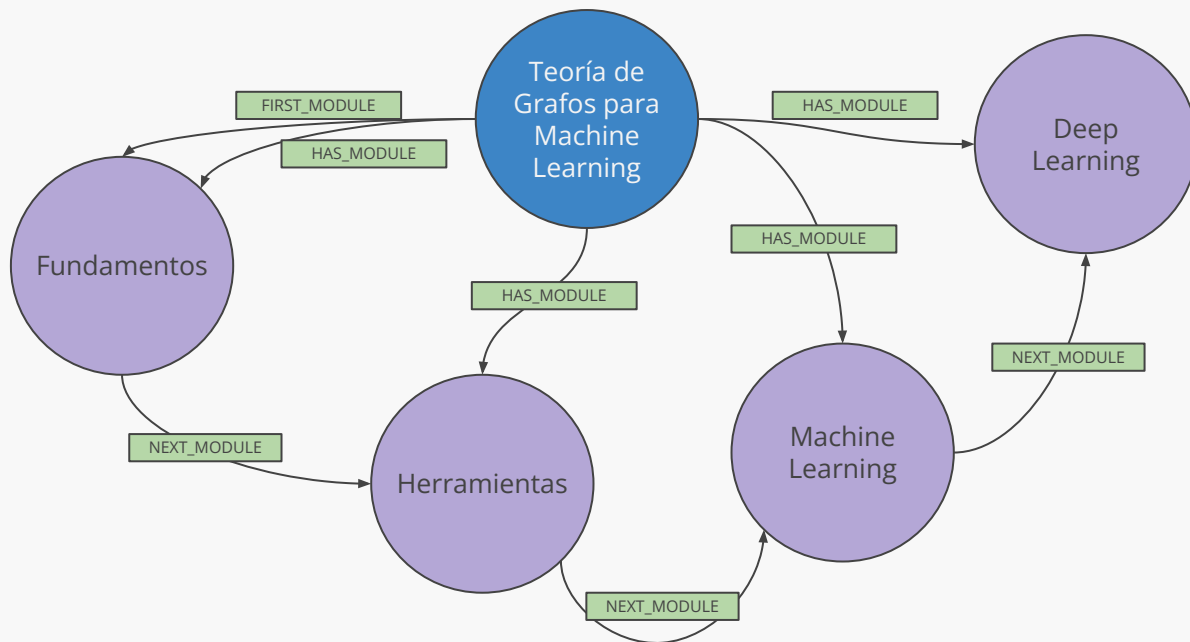


Cierre

Contenidos del curso



Contenidos del curso

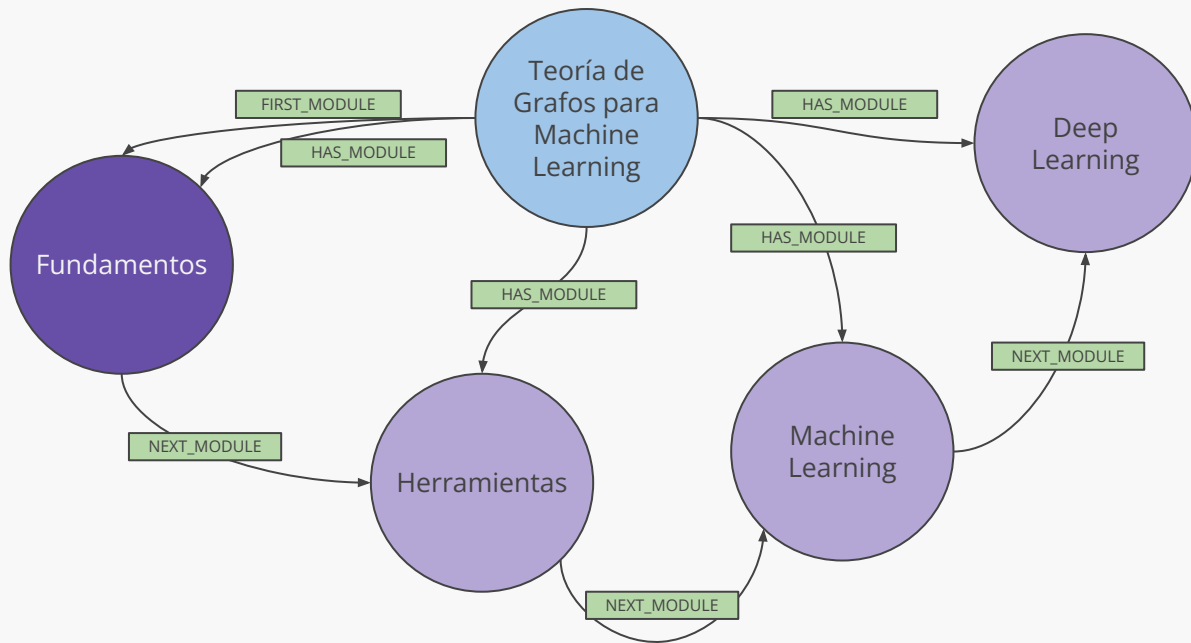


Contenidos del curso

1. Fundamentos

Desde conceptos a algoritmos fundamentales en teoría de grafos...

... para establecer una base sólida en los conceptos fundamentales de la teoría de grafos

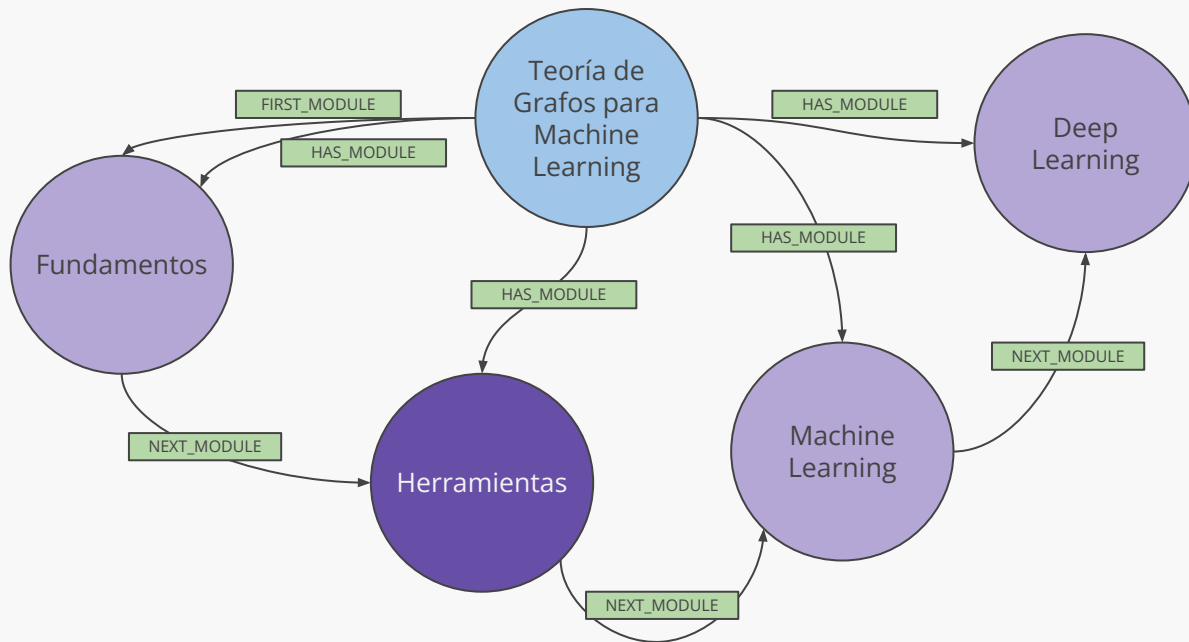


Contenidos del curso

2. Herramientas

Desde representación de pequeños grafos a analítica de grandes redes...

... para explorar y utilizar algoritmos fundamentales sobre grafos de cualquier tamaño en distintos contextos

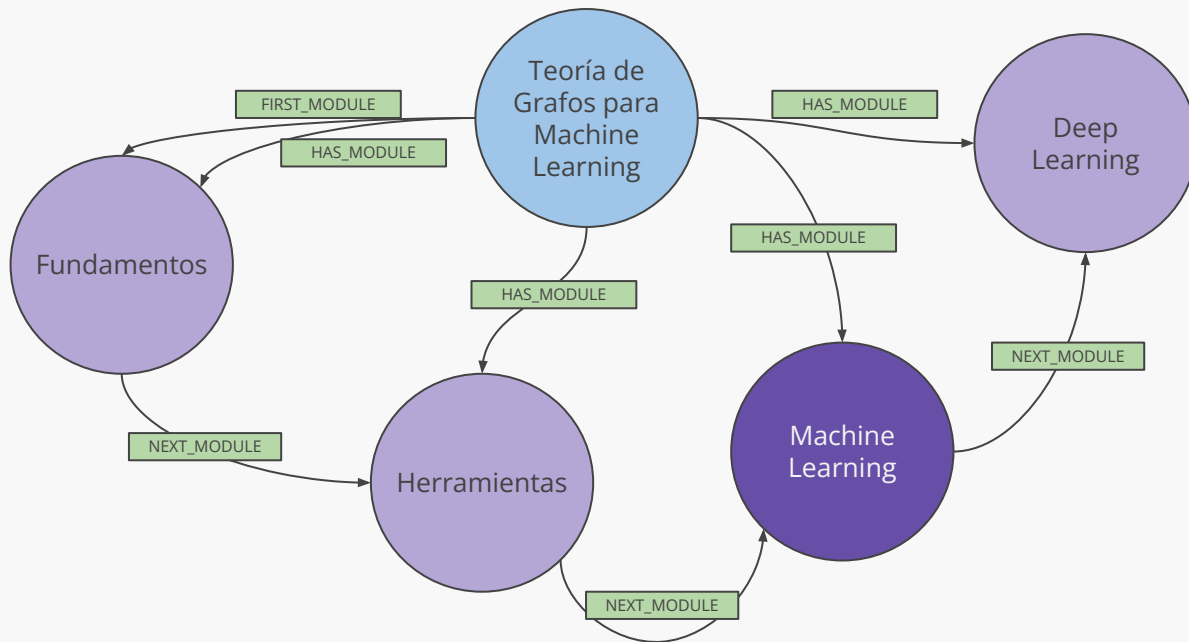


Contenidos del curso

3. Machine Learning

A partir de analítica de grafos, construir features que podamos usar en modelos de ML...

... con el objetivo de entender cómo se aplican conceptos de ML sobre grafos

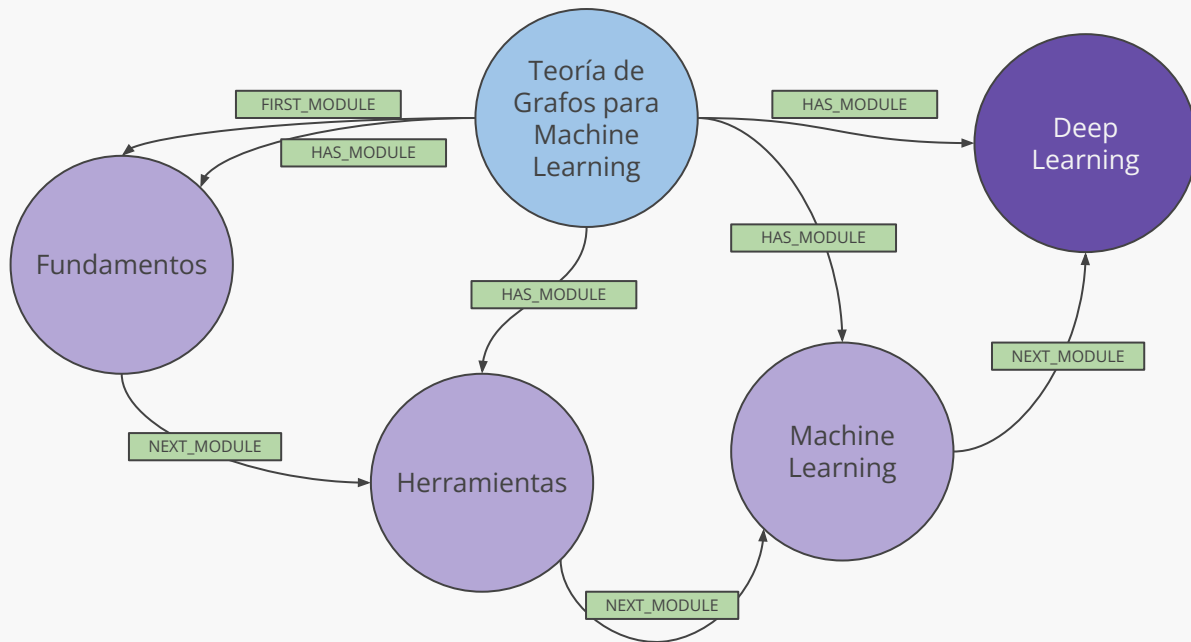


Contenidos del curso

4. Deep Learning

Explorar cómo se extiende el concepto de ML para implementar modelos especializados en grafos (GNNs)...

... de manera de aprovechar lo más posible la estructura y naturaleza de un grafo



Librerías que podrían interesarles



PyG

[PyTorch Geometric](#)

Librería para construir y entrenar GNNs, basada en **PyTorch**



Spektral

[Spektral](#)

Librería para construir y entrenar GNNs, basada en **Keras** y TensorFlow 2



[Deep Graph Library](#)

Librería para construir y entrenar GNNs, compatible con **PyTorch**, **TensorFlow** or **Apache MXNet**

Clase 8.1

Cierre del curso

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl